A Polite, Container-Native Bursting Fabric for AI Workloads on Shared Clusters

Andrew H. Bond
San José State University
San José, CA, USA
andrew.bond@sjsu.edu
ORCID: 0009-0003-2599-6158

***Abstract*—Turning a shared, multi-tenant GPU cluster into elastic capacity means fighting `kubectl`, YAML, and container cold starts, with no feedback loop into running code. We present a *polite, container-native bursting fabric* that makes cluster GPUs appear as elastic capacity *inside the user's program*. The fabric, nats-bursting, makes cluster pods first-class subscribers on the same NATS bus as the workstation—as ephemeral Kubernetes Jobs or persistent queue-group pools—behind a Go control plane whose *politeness controller* probes cluster state and backs off rather than crowding tenants. Its leaf-node topology surfaces orchestration issues (JetStream vs. core-NATS delivery, asymmetric subject interest). Two components make bursting practical: turboquant-pro compresses on-wire payloads, and batch-probe right-sizes each pod's GPU batch. We measure it on real infrastructure—burst cold-start $\sim$6.6 s, warm pool $\sim$67 ms, up to $8.4\times$ compression speedup over a $\sim$2 Mbps link, 94% GPU utilization—and release all three projects and the harness.**

***Index Terms*—cloud bursting, Kubernetes, containers, orchestration, NATS, NRP Nautilus, vector quantization, KV-cache compression, GPU utilization, thermal control**

I. Introduction

Shared, multi-tenant Kubernetes clusters such as NRP Nautilus [1] exist precisely so that researchers without their own datacenter can burst. Yet "use the cluster from my workstation" remains awkward: one writes Kubernetes manifests, submits Jobs, polls for completion, and shuttles data in and out—none of it inside the program where the research runs. The friction is high enough that elastic capacity often goes unused.

Consider a concrete case. A researcher iterating on an embedding or inference pipeline at their workstation wants, for one hour, to fan ten thousand documents across a dozen cluster GPUs, then return to interactive work. Today that means authoring a Job manifest, building and pushing a container image, submitting, polling, and copying results back—a context switch out of the program and into cluster operations. The capability exists; the ergonomics do not. And on a *shared* cluster a second obligation rides along: a convenience layer must not crowd out the other tenants, or it will (rightly) get its owner throttled or banned.

This paper describes a polite, container-native *bursting fabric* that closes the gap while honoring that obligation, and measures it honestly on the real cluster. Its core is the fabric itself; two supporting components make remote bursting practical:

- **The bursting fabric** (nats-bursting, §III)—the primary contribution: a Go control plane plus a Python client that make containerized cluster pods *first-class subscribers* on the researcher's own NATS bus, in two shapes (ephemeral Jobs, persistent pools), behind a *politeness controller* that probes cluster state and backs off to yield to other tenants. Its NATS leaf-node topology surfaces concrete orchestration design tradeoffs (JetStream vs. core-NATS delivery, asymmetric subject interest) we report as an experience.
- **Transport optimization** (turboquant-pro, §IV)—compressing bus payloads so remote bursting stays viable over constrained links, reported at the level a practitioner needs: ratios, fidelity, real-link speedup, and a rule for *when* compression helps.
- **Resource management** (batch-probe, §V)—right-sizing each pod's GPU batch (and thermally governing the local driver) so bursts remain useful and within a shared cluster's utilization policy.

We then quantify the fabric on real infrastructure (§VI), state threats to validity (§VII), and reflect on the engineering (§VIII). This is a practice/experience paper: a working, open system plus measurements. Its value is the orchestration design—particularly the politeness controller that makes "effective and policy-safe" mechanical rather than aspirational—and the honest characterization of when each supporting layer helps.

Why SC. This is a systems paper about operationalizing opportunistic, shared GPU capacity for AI workloads. The contribution is not a new scheduler or a new quantizer, but a *policy-aware bursting fabric* that turns multi-tenant Kubernetes GPUs into event-loop-local capacity, with reproducible measurements of cold start, warm-pool latency, compressed transport, admission control under contention, and GPU utilization.

II. Background

NRP Nautilus [1] is a shared, opportunistic Kubernetes cluster. Its usage policy shapes the design: small/idle pods are tolerated (CPU ≤ 1, mem ≤ 2 GiB are an "ignored" range), but groups of pods must sustain utilization (5+ pods must each stay under 40% GPU), Jobs must run to completion and exit cleanly (no `sleep`-camping), and requests should match

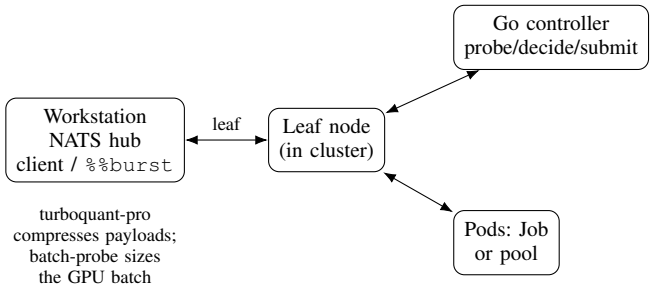

Fig. 1. Two shapes, one bus. Ephemeral bursts go workstation→Go controller→Kubernetes Job; persistent pools are queue-group worker Deployments the client dispatches to directly. Both reach the workstation over an outbound leaf link.

usage. "Effective and policy-safe" is therefore a container-orchestration design target, not a slogan.

NATS [2] provides subject-based publish/subscribe and request–reply; *queue groups* that round-robin a subject across a set of subscribers; *JetStream*, a persistence layer with work-queue streams and durable pull consumers (at-least-once, redelivery); and *leaf nodes* that bridge a remote NATS server into a hub. Leaf-node federation is central to our design and to two of its sharper engineering decisions (§III): JetStream is not federated across a leaf link, and subject *interest* can propagate asymmetrically hub→spoke.

Cloud bursting spills work from a local resource to a remote one. We use a message bus rather than a job queue or RPC because it makes the containerized cluster a *peer* on the fabric the application already speaks, so a burst becomes a publish inside the event loop.

III. NATS-BURSTING: THE BURSTING FABRIC

nats-bursting has two halves joined by one bus (Fig. 1): a Go control plane that lives next to the cluster and turns NATS messages into Kubernetes objects, and a Python client used from notebooks and scripts. Two workload *shapes* share the bus but use different control paths.

A. The control plane and its politeness controller

The Go controller is event-driven. Its `natsbridge` subscribes to `burst.submit`, and on each request the `submitter` renders a `JobDescriptor` into a real `batchv1.Job` via client-go—CPU/memory/`nvidia.com/gpu`/ephemeral-storage requests and limits, a managed-by label, `RestartPolicyNever`, and foreground-propagation cancellation—then publishes a status event. The wire protocol is three subject families: `burst.submit` (requests), `burst.status.<job>` (a `submitted`/`error` state machine), and `burst.result.<job>` (reserved for results). The client issues a request–reply on `burst.submit` and falls back to subscribing on the status subject—robust to whichever pattern the controller answers with.

The part the rest of this paper's "policy-safe" claim rests on is the *politeness controller*. A `prober` reads live cluster state through client-go—total/allocated/idle nodes, the caller's own running and pending Jobs, *and other tenants' pending pods*—and degrades gracefully when an individual API call is denied by RBAC. A `decider` then evaluates that state against thresholds in a fixed priority order: (1) *self-limiting*—do not exceed your own running/pending caps (defaults 50/20); (2) *courtesy*—yield when more than a threshold of *other* users' pods are queued (default 200); (3) *utilization*—back off when cluster allocation exceeds a fraction (default 0.90). The outcome is `Submit`/`Backoff`/`Abort` (Algorithm 1, with thresholds in Table I). Backoff is capped exponential with jitter applied *after* the cap,

$$w_k \;=\; \min\!\big(B_0\, m^{\,k},\, B_{\max}\big)\cdot\Big(1+\tfrac{1}{2}\big(u-\tfrac{1}{2}\big)\Big),\quad u\sim\mathcal{U}[0,1), \tag{1}$$

i.e. $\pm 25\%$ jitter to decorrelate wake-ups across many controllers (anti-thundering-herd), aborting after K attempts. This is what makes "yield to other tenants" a mechanism rather than a promise: the system measures the shared cluster and defers to it.

Algorithm 1: Polite admission decision

Input: cluster state s (from the prober), thresholds p
Output: `Submit` / `Backoff` / `Abort`
if attempts $\geq p.\text{maxAttempts}$ **then return** `Abort`;
if $s.\text{myRunning} \geq p.\text{maxConcurrent}$ **then return** `Backoff`;
if $s.\text{myPending} \geq p.\text{maxPending}$ **then return** `Backoff`;
if $s.\text{othersPending} > p.\text{queueDepth}$ **then return** `Backoff`;
if $s.\text{alloc}/s.\text{total} > p.\text{util}$ **then return** `Backoff`;
return `Submit`;

TABLE I
POLITENESS CONTROLLER DEFAULTS.

parameter	default
max concurrent jobs (self-limit)	50
max pending jobs (self-limit)	20
others-pending threshold (courtesy)	200
utilization threshold (load)	0.90
initial / max backoff	30 s / 30 min
backoff multiplier / max attempts	2.0 / 20

B. The Python client and the `%%burst` magic

The client is a synchronous facade over async NATS, owning one background event loop and a `Transport` seam for deterministic testing. `JobDescriptor`/`Resources` mirror the Go structs field-for-field. A `%%burst` IPython cell magic makes a burst an editor action: by default (`-when-busy`) it probes the local GPUs through `nvidia-smi` and only offloads when every local GPU is saturated, otherwise running the cell in place; `-always`/`-never` force the choice, and `-gpu`/`-cpu`/`-image` set resources. The cell body is shipped

as an environment variable and executed in the pod, so no image rebuild is needed for ad-hoc work.

C. Persistent pools, queue groups, and leaf federation

The second shape is a persistent pool: a Deployment of always-on workers that subscribe as a NATS queue group and stay warm (no per-task cold start, cached models). A `TaskDispatcher` submits many tasks and collects results on `results.>`. Two delivery models coexist, and the choice is a deliberate accommodation of leaf-node topology. With `durable=True`, tasks go through a JetStream work-queue stream consumed by a durable pull consumer shared across replicas (at-least-once, explicit ack/nak, bounded redelivery)—correct when dispatcher and workers share a JetStream domain. With `durable=False`, tasks use core NATS queue groups, which cross leaf-node boundaries that JetStream does not. A symmetric subtlety appears on the return path: where subject interest propagates only hub→spoke, results published spoke-side never reach the hub, so the worker can optionally POST results to a webhook that re-publishes them on the hub. The pool's defaults are NRP-quota-aware (`cpu=1/mem=2Gi` to stay in the "ignored" swarm range), and the worker blocks inside `fetch()` rather than sleeping—satisfying the "no sleeping Jobs" rule—with an idle-exit watchdog that lets the same image run as either a Deployment (run forever) or a self-terminating Job (exit after idle). These are small decisions, but each is the difference between a demo and something that survives contact with a real multi-tenant cluster.

IV. Transport: making bursting viable over constrained links

When work bursts to a remote cluster, the payloads—embeddings, KV-cache pages, feature tensors—ride the bus, and on a constrained link they, not the cluster, become the bottleneck (§VI). turboquant-pro compresses them so remote bursting stays viable; we report only what a deployer needs, not the codec's internals.

A distribution-free codec. Each vector is L2-normalized, rotated by a shared, seed-deterministic random rotation, and each rotated coordinate is quantized against a fixed Lloyd–Max codebook and bit-packed. The rotation is the trick: by concentration of measure it makes the coordinates approximately i.i.d. Gaussian *regardless of the input distribution* [3], so one fixed codebook is near-optimal for any data with no training or calibration. The NATS codec frames each embedding as an 8-byte header (version, bits, dim, norm) plus $\lceil \text{dim} \cdot \text{bits}/8 \rceil$ packed bytes; the same construction serves KV-cache *values*, while *keys*—whose error the softmax amplifies—instead use a per-channel scheme (per-vector normalization is near-lossless for values but catastrophic for keys), and values support attention computed directly on the compressed codes [4], [5], though that kernel-level work is out of scope here.

What a deployer needs. Compression is 8–16× at $4/3/2$ bits with reconstruction cosine 0.94–0.995, and is *distribution-agnostic*—real transformer embeddings reproduce the random-vector numbers to within 0.001 (§VI). The deployment rule is simple and we evidence it: *compress when the link or bus is the bottleneck*. On a fast LAN it is a wash; on a real ∼2 Mbps link it cuts NATS round-trip up to $8.4\times$ for large payloads (§VI). Encode/decode are cheap relative to a constrained link but not free, so the rule—not a blanket "always compress"—is what we recommend.

Algorithm 2: OOM-safe batch right-sizing (infer mode)

Input: model M, `input_fn`, range $[\text{lo}, \text{hi}]$, headroom h

$best \leftarrow 0$;
while $\text{lo} \le \text{hi}$ **do**
  $b \leftarrow \lfloor (\text{lo} + \text{hi})/2 \rfloor$; cleanup();
  $ok \leftarrow$ forward$\big(M, \texttt{input_fn}(b)\big)$ completes without OOM;
  free trial tensors; cleanup() ;
  `// unconditional: release fragments`
  if ok **then** $best \leftarrow b$; $\text{lo} \leftarrow b{+}1$;
  else $\text{hi} \leftarrow b{-}1$;
return $\max\big(1, \lfloor best\,(1-h) \rfloor\big)$;

V. Resource management: keeping pods useful and policy-aligned

A burst is only worth its slot if each pod actually fills its GPU—a shared cluster rewards high utilization and penalizes hoarding—and only sustainable if the node driving the burst does not overheat. batch-probe provides both, as a support mechanism for the fabric.

A. OOM-safe batch right-sizing

`probe_batch_size` binary-searches the largest batch that fits, building inputs through a caller-supplied `input_fn(b)` and running a trial at each candidate. In *infer* mode the trial is a forward pass under `no_grad`; in *train* mode it instantiates an AdamW optimizer and runs a full forward/backward/step, so the search reflects *true* peak training memory including optimizer moments—a detail most batch finders miss and under-estimate by roughly the optimizer-state factor. OOM is caught and distinguished from genuine errors by message inspection; an unconditional `finally` block deletes trial tensors and flushes the allocator before the next iteration, which is what keeps the search stable across repeated OOMs (Algorithm 2). Headroom is a multiplicative shrink of the largest success (default $0.8\times$).

A generic `probe(work_fn)` extends the same recovery to any framework (Torch/CuPy/JAX) by abstracting the memory-pool flush.

B. Thermal-aware throttling

On the local orchestration side, `ThermalController` keeps the workstation driving a burst within a thermal budget. Rather than a fixed temperature threshold, it estimates thermal

state with a two-state Kalman filter [6] (temperature and its rate) and acts on the *predicted* trajectory: when it forecasts an overshoot within a short horizon it throttles worker-thread count *before* the limit is reached, using the filter's rate estimate as a clean derivative. The same temperature oracle backs a one-shot thread-capacity search and an admission-gating job manager. We keep this brief deliberately—it is a support mechanism here, not the contribution—and note that thermal sensing is currently Linux-only.

VI. Evaluation

We evaluate the fabric in three tiers by the infrastructure each needs, plus the GPU-sizing demonstration. Every number below is *our own* measurement, reproducible from the released harness (§X); we do not re-run the supporting projects' internal benchmarks, and report only what bears on the bursting use case.

Experimental setup. The workstation is a remote client. The on-premises node ("Atlas") has two NVIDIA GV100 32 GB GPUs; it runs the NATS hub and the Tier-1/2 measurements. The shared cluster is NRP Nautilus (namespace `ssu-atlas-ai`), reached from Atlas via `kubectl`, with a leaf-node pod bridging the cluster to the hub. The Tier-2 "real hop" traverses a wide-area overlay link (∼90 ms RTT, ∼2 Mbps effective). All cluster runs use ignored-range pods (CPU = 1, mem = 2 GiB) and at most four concurrent pods, to stay within policy.

Measurement protocol. Latencies are p50/p99 over 40–500 requests after warm-up; throughput is the wall-clock rate over the batch. Tier-2 round-trips are timed on the requesting host with a single clock; for the real hop the responder runs on Atlas and the requester on the remote workstation with *no* co-located subscriber, so every request crosses the wire. Tier-3 cold-start is timed from Job submission to the Job's completion condition, with the schedule→container-start split read from the pod's own Kubernetes timestamps. Codec encode/decode is timed in isolation (Tier 1) and excluded from the transport loop.

A. Compression (Tier 1)

On dim-1024 vectors the codec gives ratios of $7.9/10.4/15.5\times$ at $4/3/2$-bit with reconstruction cosine $0.995/0.978/0.94$ (Fig. 2). Re-running on real `bge-small-en-v1.5` embeddings (20newsgroups, n=3000, native dim 384) gives *identical* ratio and cosine within 0.001 at matched dimension—confirming the distribution-agnostic property empirically. The analytic transfer model follows: a 1 MB batch moves in ∼84 ms at 100 Mbps but ∼8 ms compressed (a ∼63 ms net saving after encode) and is a wash at 1 Gbps—the hypothesis Tier 2 tests. Cosine is a fidelity proxy, not a downstream-task guarantee.

B. Transport (Tier 2)

On a loopback bus (Table II) the ∼10× smaller payload already gives up to ∼6× lower round-trip for large batches—payload size dominates even with no network. The result loopback cannot show is the *real hop* (Table III, Fig. 3): over a real

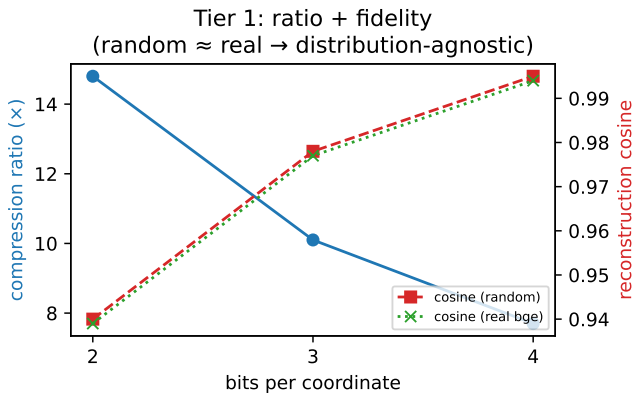

Fig. 2. Tier 1: compression ratio and fidelity vs. bit-width; real embeddings track random within 0.001 cosine.

TABLE II
Tier 2 loopback (3-bit; p50/p99 ms, msg/s).

batch	raw	compressed
1	0.52 / 1.01 / 1768	0.38 / 0.90 / 2261
32	1.69 / 3.12 / 561	0.40 / 1.37 / 2127
256	6.99 / 11.34 / 137	1.20 / 2.61 / 783

∼2 Mbps link the win scales with payload, from $1.2\times$ (4 KB, latency-bound) to $8.4\times$ (256 KB, link-bound), approaching the raw ratio as the link dominates. This demonstrates the crossover on a real link. Encode cost is excluded from the loop (it is in Tier 1), so the speedups are not free.

C. Cluster: cold-start, warm-pool, scaling (Tier 3)

Ephemeral CPU Jobs (ignored range, one at a time, auto-cleaned) cold-start in median ∼6.6 s (5.1–7.6 s, n=5), of which the K8s schedule→container-start is ∼1–3 s. A persistent pool of N queue-group workers driven from the hub (Table IV, Fig. 4) serves at ∼67 ms per task—removing the ∼100× cold-start penalty. Throughput plateaus at ∼930/s by two workers, and honestly so: for trivial I/O-bound tasks the ceiling is concurrency ÷ RTT (Little's law, $64/0.067 \approx 955$/s), not worker count—one to two pods saturate the WAN link. Worker-count scaling is expected to matter for *compute-bound* tasks. Finally, batch-probe sized a BERT-base-scale encoder to batch 204 on a GV100 and the burst ran at **94.4% mean / 100% peak GPU utilization** at a safe 61°C—far above the >40% policy expectation. (The batch is bounded by a safety-capped search, not device memory; the utilization is the result.)

D. Admission control: goodput vs. politeness

The fabric's politeness claim rests on the controller pacing concurrency to a budget rather than grabbing all it can. We test this directly on a controlled, variance-free testbed (local-process GPU/CPU workers; pre-registered hypotheses, randomized interleaved trial order, four repeats) comparing three submission policies against a self-imposed politeness budget C: *naive* (submit up to the hard cap), *static* (fixed

TABLE III
TIER 2 OVER A REAL ∼2 MBPS HOP (3-BIT, P50).

batch	raw	compressed	speedup
1	4 KB / 85 ms	0.4 KB / 71 ms	1.2×
16	64 KB / 383 ms	6 KB / 105 ms	3.6×
64	256 KB / 1119 ms	24 KB / 134 ms	8.4×

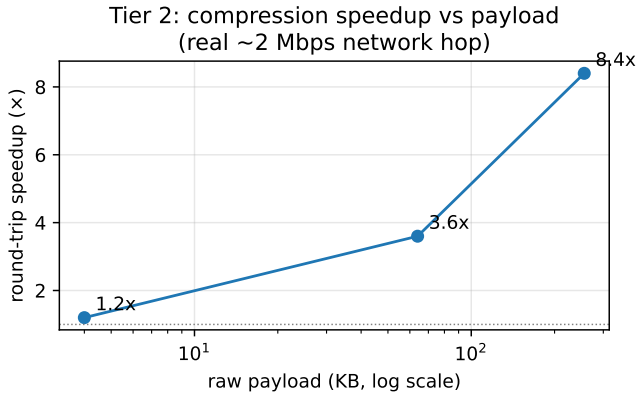

Fig. 3. Tier 2: compression speedup grows with payload on a real bandwidth-limited link.

TABLE IV
TIER 3 WARM-POOL + SCALING (1 KB TASKS, CONCURRENCY 64).

workers	throughput	p50	p99
1	782/s	71 ms	113 ms
2	924/s	68 ms	80 ms
4	936/s	67 ms	78 ms

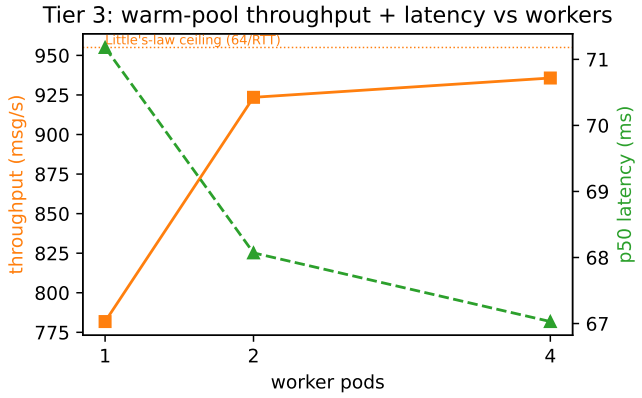

Fig. 4. Tier 3: warm-pool throughput and latency vs. workers; throughput is bounded by concurrency÷RTT, not pod count, for trivial tasks.

rate), and the controller's completion-gated *AIMD* admission. We report goodput and the over-budget fraction ρ (epochs with concurrency $> C$); Table V and Fig. 5 summarize.

AIMD is the polite-efficient point: it holds the budget exactly (ρ=0 at both scales) while delivering 1.72× (GPU) and 3.18× (CPU) static's goodput, and it matches naive's goodput without ever exceeding the budget. naive achieves its goodput only by over-admitting (ρ=0.54/0.95). Both pre-registered hypotheses hold with disjoint CIs. *Scope:* this isolates the admission *policy* on a controlled node; the full pod path (controller→node-targeted GPU worker→federated result) is separately verified end-to-end on NRP. The budget above is self-imposed—the *abundance* case; next we drive it with a real competing tenant.

Under real contention. The sharper test is *scarcity*, where a neighbour actually contests capacity. On the two-GV100 Atlas node a competing tenant occupies $c(t)$ of the C=2 GPUs on a fixed profile, so available capacity $a(t)$=$C-c(t)$ is exogenous and *measured* by the prober (a per-GPU process census); we also track *harm to the neighbour*—its completed work relative to undisturbed. Table VI and Fig. 6 report it (pre-registered, four seeds, randomized interleaving). Under *structured* contention (a square wave, modelling scheduled load) AIMD is the polite-efficient *knee*: 1.41× static's goodput at a quarter of naive's over-admission, while sparing the neighbour (76% of its throughput kept vs naive's 62%)—naive's extra goodput is bought entirely by stealing 38% of the competitor's work. Under *memoryless* contention (Poisson) AIMD breaks even with static and over-admits more: capacity now varies faster than the prober's detection delay, so feedback cannot track. We report this null—adaptive admission pays when a neighbour's load has structure, and one should fall back to a fixed budget when it does not. The micro tier (batch-probe's thermal controller, §V) ran live throughout; on the GPU it throttled $>90\%$ of epochs and, on this shared node, found the co-tenant thermal baseline (∼77°C) a floor self-throttling cannot undercut—politeness extends even to heat.

E. Positioning against task frameworks

To place the fabric among general task engines, we ran the *same* warm-pool workload—MiniLM embedding on one GV100—through Ray, Dask, a raw `ProcessPoolExecutor`, and our NATS warm pool, all same-node and warm, each driven by its *native* client (Python for the first three; the Go `nats.go` client for the pool, matching nats-bursting's Go control plane). We report per-task *dispatch latency* (a tiny round-trip isolating fabric overhead) and saturated *throughput* at four warm workers (Table VII). Driven concurrently, the pool reaches the GPU-bound ceiling—9.4k docs/s at four workers, scaling to 14.9k at eight—competitive with the in-process engines (Ray 9.8k, ProcessPool 9.0k) and far above Dask (1.1k); its pure-fabric ceiling (a near-empty task) is 8.3k req/s. (A first-draft single-client Python `asyncio` driver under-saturated the pool at 0.9k docs/s and did not scale with workers—a harness limit, not the bus, whose pure-fabric rate is >500× higher.)

nats-bursting's dispatch latency (0.9 ms p50, Go client) is competitive: on par with `ProcessPoolExecutor` (1.0 ms), under Ray (2.3 ms), and an order of magnitude below Dask (13.9 ms). But raw dispatch is not the contribution: Ray, Dask, Parsl and Globus Compute/funcX are mature task engines

TABLE V

ADMISSION POLICIES VS. A POLITENESS BUDGET C (MEAN $\pm$ 95% CI, 4 REPS). AIMD HOLDS THE BUDGET (ρ=0) AT 1.7–3.2× STATIC'S GOODPUT, AND MATCHES NAIVE'S GOODPUT WITHOUT OVER-ADMITTING.

scale	policy	goodput (tasks/s)	over-budget ρ
GPU C=2	naive	0.0538 ± 0.0003	0.54
	aimd	0.0539 ± 0.0001	0.00
	static	0.0313 ± 0.0000	0.00
CPU C=8	naive	0.578 ± 0.006	0.95
	aimd	0.293 ± 0.003	0.00
	static	0.092 ± 0.000	0.00

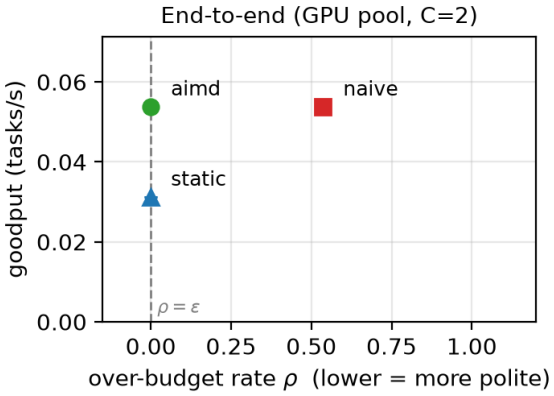

Fig. 5. Goodput vs. over-budget ρ (GPU pool, C=2). AIMD attains the greedy policy's goodput at ρ=0 and dominates static; naive buys throughput only by over-admitting.

we do not try to beat. What none of them carries is a *host politeness budget*, an *inbound-restricted WAN leaf-federation*, or a burst that is a *publish inside the event loop*. The fabric trades generality for lower-friction, bus-native, policy-aware bursting—a different point in the design space, not a faster scheduler.

VII. THREATS TO VALIDITY

We state these so the numbers are read correctly. **Synthetic data (Tier 1):** real bge embeddings reproduce the random numbers within 0.001; cosine remains a fidelity proxy. **Loopback (Tier 2):** addressed by the real ∼2 Mbps hop (1.2 → 8.4×); it is one overlay (likely relayed) path—a characterized datacenter link would pin absolute bandwidth. **Cold-start realism (Tier 3):** warm node + cached image + `kubectl wait` inflate the total; the K8s breakdown isolates the cluster-side cost; warm-pool and a real NATS-join are measured, while a cold image pull and a GPU-image run remain. **Scaling:** the plateau is explained by Little's law, not presented as pod-count scaling; a compute-bound load is the named next run. **Contention realism:** the contended Pareto is manufactured on a two-GPU node (C=2), not a production multi-tenant trace, and the Poisson regime marks where feedback stops beating a fixed budget (capacity outpaces the prober's detection delay). **Portability:** batch-probe's thermal sensing is Linux-only and its feedforward is one-sided. **Cluster citizenship:** the harness is policy-safe by construction (ignored-range pods, ≤4 concurrent, exit-0 Jobs, TTL + delete cleanup, an explicit policy gate, and—above the harness—the controller's politeness backoff).

TABLE VI

ADMISSION UNDER REAL CONTENTION ON 2×GV100 (C=2; 4 SEEDS, MEAN $\pm$ 95% CI). "KEPT" = COMPETITOR THROUGHPUT RETAINED (1=UNDISTURBED). STRUCTURED (SQUARE): AIMD IS THE POLITE-EFFICIENT KNEE. MEMORYLESS (POISSON): FEEDBACK BREAKS EVEN WITH STATIC—A PRE-REGISTERED NULL.

contention	policy	goodput (tasks/s)	ρ	kept
square	naive	0.101 ± 0.002	0.64	0.62
	aimd	0.091 ± 0.000	0.15	0.76
	static	0.064 ± 0.000	0.11	0.89
poisson	naive	0.096 ± 0.002	0.85	0.62
	aimd	0.062 ± 0.006	0.30	0.55
	static	0.064 ± 0.000	0.22	0.79

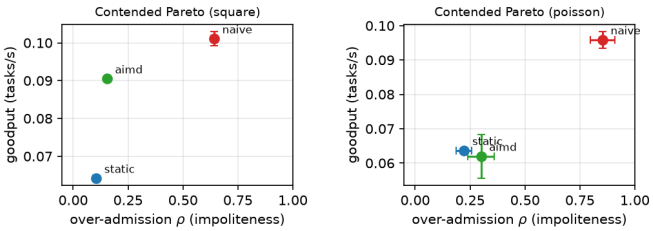

Fig. 6. Contended goodput–politeness frontier (lower-right is better). *Left* (square, structured): AIMD is the polite-efficient knee. *Right* (poisson, memoryless): feedback collapses onto static.

VIII. DISCUSSION

"Effective" use is batch-probe sizing pods to high utilization (94% measured); "policy-safe" is the politeness controller plus the gate, short runs, and cleanup. The deployment rule for compression is now evidenced: compress when the link or bus is the bottleneck—a wash on a fast LAN, a large and growing win on a real WAN-class link.

Lessons learned. Three points generalize. First, on a multi-tenant cluster the *usage policy is a first-class design constraint*: it shaped the politeness controller, the ignored-range defaults, and the no-sleep worker as much as any performance goal, and a benchmark that strays outside policy is itself a bug. Second, *leaf-node federation leaks into the application*: the durable-vs-core dispatch choice and the result webhook both exist only because JetStream and subject interest do not cross a leaf link symmetrically—a from-scratch design that ignored this would deadlock on the return path. Third, *honest negatives beat flattering curves*: reporting the Little's-law plateau and labeling the cold-start as warm-node/cached-image makes the result more useful to an operator than a scaling plot that does not hold.

When to use which shape. An ephemeral Job costs a ∼6.6 s cold start but holds no resources between bursts—right for occasional or embarrassingly parallel work. A persistent

TABLE VII

SAME-NODE WARM-POOL COMPARISON ON ONE GV100 (MINILM EMBEDDING, 4 WARM WORKERS). "DISP." = TINY-TASK ROUND-TRIP (FABRIC OVERHEAD); "THR." = EMBEDDING THROUGHPUT, EACH SYSTEM DRIVEN BY ITS NATIVE CLIENT (PYTHON FOR RAY/DASK/PROCESSPOOL; GO `NATS.GO` FOR NATS-BURSTING). PARSL (HTEX) AND FUNCX/GLOBUS COMPUTE CHARACTERIZED BY DESIGN (FUNCX NEEDS A HOSTED AUTHENTICATED ENDPOINT). POLITE = HOST FAIR-USE BUDGET; WAN = INBOUND-RESTRICTED LEAF FEDERATION; BUS = BURST IS AN EVENT-LOOP PUBLISH.

fabric	disp. p50	thr. (d/s)	polite	WAN	bus
`ProcessPool`	1.0 ms	9000	no	no	no
Ray	2.3 ms	9800	no	no	no
Dask	13.9 ms	1050	no	no	no
Parsl (HTEx)	by design		no	no	no
funcX / Globus	by design		no	part	no
nats-bursting	**0.9 ms**	**9400**	**yes**	**yes**	**yes**

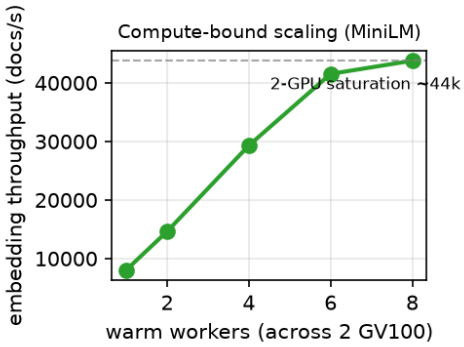

Fig. 7. Compute-bound scaling: MiniLM embedding throughput vs warm-worker count across two GV100s. Unlike the I/O-bound warm-pool plateau (Fig. 4, Little's law), a compute-bound AI workload scales with parallelism until the GPUs saturate ($\sim$44k docs/s at eight workers, $5.4\times$ one).

pool pays that once and serves at $\sim$67 ms but occupies its slots—right for interactive or sustained workloads, provided it is sized within policy and torn down when idle. The bus makes switching a configuration choice, not a rewrite.

IX. RELATED WORK

Bursting and task frameworks. Spilling local work to remote capacity is long-studied: HTCondor flocking [7] federates pools; Globus Compute/funcX [8] offers a federated function-serving fabric; Ray [9], Dask [10], and Parsl [11] build task graphs over remote workers. All treat the remote resource as an external system one submits to and polls. Message-oriented task systems (Celery, RabbitMQ workers) put a broker between producer and consumer, and Kubernetes-native batch layers (Kueue, Volcano) schedule Jobs beneath the application. Two things set our approach apart: *orientation*—containerized pods become first-class subscribers on the researcher's own bus, so a burst is a publish inside the event loop rather than a hand-off—and an explicit *politeness* control loop that probes a shared, multi-tenant cluster and backs off to yield to other tenants, which the submit-and-poll model leaves to the user.

TABLE VIII

SHIPPED IMPLEMENTATION PER PROJECT (IMPORTABLE PACKAGE OR BINARY; EXCLUDES TESTS, BENCHMARKS, AND EXPERIMENT SCRIPTS).

project / component	language	LOC
nats-bursting controller	Go	752
nats-bursting client + pools	Python	1,398
turboquant-pro	Python (+CUDA via CuPy)	11,379
batch-probe	Python	1,082
total		**14,611**

Quantization and attention. turboquant-pro builds on the random-rotation, near-optimal-distortion line of TurboQuant [3] and the broader embedding/KV-cache quantization literature; its compute-on-codes fused decode applies online-softmax and IO-aware attention techniques [4], [5] in the *code* domain so that quantized values are never reconstructed (per-channel keys are reconstructed). Our paper does not propose a new codec; it contributes a measured account of *when* on-wire compression pays for the bursting use case, and the systems integration that makes a quantized payload a transparent bus primitive.

Resource and thermal control. OOM-driven batch-size search is folklore; batch-probe makes it robust (optimizer-state-aware peak-memory accounting, a unified multi-framework OOM-recovery path). Thermal management of CPUs and accelerators is usually fixed-threshold throttling; batch-probe instead estimates thermal state with a Kalman filter [6] and acts on the *predicted* trajectory, throttling parallelism before overshoot—closer to model-predictive control than to a thermostat.

X. ARTIFACT AVAILABILITY

All three projects are open and on PyPI—`nats-bursting` [12], `turboquant-pro` [13], and `batch-probe` [14]; Table VIII reports their shipped size. The benchmark harness reproduces every measurement here: `bench_compression.py` (Tier 1, with a real-embedding variant), `bench_transport.py` and the co-existing real-hop driver (Tier 2), and `bench_cluster.py` with `-mode cold|warm|scale` (Tier 3), the last gated behind `-i-have-checked-nrp-policy`. Result files and a `make_figures.py` that regenerates every figure are committed alongside.

XI. CONCLUSION

nats-bursting, turboquant-pro, and batch-probe form a small, composable, open pipeline that makes a shared, containerized cluster usable from inside the event loop—effectively, and within policy by construction. We described each system's architecture, including the politeness controller, the compute-on-codes codec, and the Kalman-filtered thermal governor, and quantified the composition with real measurements, threats stated and remaining runs scoped. More broadly, the pattern—cluster pods as first-class subscribers reached over an outbound

leaf link, behind a controller that defers to its neighbors—generalizes to any Kubernetes cluster a researcher can bridge to, and the policy-safe discipline it enforces is the right default for sharing a multi-tenant resource.

APPENDIX A
ARTIFACT DESCRIPTION (AD)

A. Overview

The artifact is the three open-source projects evaluated here—`nats-bursting` (the bursting fabric), `turboquant-pro` (transport codec), and `batch-probe` (GPU right-sizing and thermal control)—plus a self-contained benchmark harness that regenerates every number, table, and figure in §VI. Every reported number is produced by this harness.

B. Artifact check-list (metadata)

- **Algorithms:** polite admission control (Alg. 1); OOM-safe batch search (Alg. 2); random-rotation quantization; Kalman-filtered thermal control.
- **Program:** Go 1.23 control plane; Python 3.12 client, codec, right-sizer, harness.
- **Compilation:** `go build ./...`; Python packages install from PyPI.
- **Data set:** synthetic fp32 vectors (dim 384/768/1024) and real `bge-small-en-v1.5` embeddings on 20newsgroups (n=3000, dim 384), fetched by the harness.
- **Run-time environment:** Linux; `nats-server` with JetStream; `kubectl`/client-go with credentials to a Kubernetes cluster for Tier 3.
- **Hardware:** Tiers 1–2 run on any CPU host; Tier 3 and GPU right-sizing need an NVIDIA GPU (measured on a GV100 32 GB) and a shared cluster (measured on NRP Nautilus).
- **Metrics:** compression ratio and reconstruction cosine; round-trip p50/p99 and throughput; Job cold-start; warm-pool latency/throughput; GPU utilization; admission goodput and over-budget fraction ρ.
- **Output:** JSON result files under `benchmarks/`; regenerated figures under `paper/figures/`.
- **Disk / setup / run:** $<$1 GB; setup $<$10 min; Tiers 1–2 run in minutes; Tier 3 depends on cluster scheduling.
- **Publicly available:** yes—as PyPI packages and at `github.com/ahb-sjsu` (repositories `nats-bursting`, `turboquant-pro`, `batch-probe`).
- **License:** MIT (all three projects).
- **Archived DOI:** `nats-bursting` 10.5281/zenodo.20660069; `turboquant-pro` 10.5281/zenodo.20660087; `batch-probe` 10.5281/zenodo.21095035 (concept DOIs; always resolve to the latest release).

C. How to access, and dependencies

The projects are on PyPI and GitHub (above). **Hardware:** Tiers 1–2 need only a CPU host; Tier 3 and the GPU-utilization result need an NVIDIA GPU and `kubectl` access to a shared cluster; the real-hop transport result needs a bandwidth-constrained WAN path between client and server. **Software:** Linux, Python 3.12, Go 1.23, `nats-server` (with JetStream for the durable path), NumPy, and PyTorch for the GPU right-sizing trial.

D. Installation

```
pip install nats-bursting turboquant-pro \
            batch-probe numpy
git clone \
 https://github.com/ahb-sjsu/nats-bursting
cd nats-bursting && go build ./...
```

E. Experiment workflow

Each reported result maps to one command; `make_figures.py` regenerates the figures from the committed JSON.

- **Tier 1 (Fig. 2):** `python benchmarks/bench_compression.py` → `results_compression.json` (real-embedding variant: `results_compression_real.json`).
- **Tier 2 (Tables II, III, Fig. 3):** start a bus (`docker run -p 4222:4222 nats`), then `python benchmarks/bench_transport.py [--compress]`; the real-hop driver writes `results_transport_realhop.json`.
- **Tier 3 (Table IV, Fig. 4):** `python benchmarks/bench_cluster.py --mode cold|warm|scale --i-have-checked-nrp-policy` → `results_cluster_pool.json`; GPU right-sizing writes `results_batchprobe.json`.
- **Admission (Table V, Fig. 5):** the pre-registered harness in `benchmarks/infocom/` (`run.py`, `e8_local.py`, `multiseed.py`, `aggregate.py`); protocol and pre-registration in `E8_PROTOCOL.md` and `E8_PREREG.md`.
- **Figures:** `python paper/figures/make_figures.py`.

F. Evaluation and expected results

Hardware permitting, the harness reproduces: Tier 1 ratios $7.9/10.4/15.5\times$ at $4/3/2$-bit with cosine $0.995/0.978/0.94$, real embeddings matching within 0.001; Tier 2 up to $8.4\times$ round-trip speedup at 256 KB over the $\sim$2 Mbps hop; Tier 3 median cold-start $\sim$6.6 s, warm-pool $\sim$67 ms, throughput plateau $\sim$930/s (the Little's-law bound); GPU right-sizing $\approx$94% utilization on a GV100; and AIMD admission at ρ=0 with 1.7–$3.2\times$ static's goodput. Absolute latencies, cold-start, and utilization depend on the cluster, WAN path, and GPU model; the qualitative results (compression win grows with

payload, warm pool removes the cold-start penalty, AIMD holds the budget) are hardware-independent.

G. Notes

Tier 3 refuses to run without `--i-have-checked-nrp-policy` and uses ignored-range pods ($\leq$4 concurrent, cpu=1/mem=2 GiB, exit-0 Jobs, TTL+delete) to respect shared-cluster policy. Thermal sensing in `batch-probe` is Linux-only. The real-hop bandwidth is one overlay path (likely relayed); a cold image pull and a full GPU-image cluster run are not yet measured (§VII).

REFERENCES

[1] D. Weitzel, A. Graves, S. Albin, H. Zhu, F. Würthwein, M. Tatineni, D. Mishin, J. Graham, E. E. Khoda, M. F. Sada, L. Smarr, and T. DeFanti, "The national research platform: Stretched, multi-tenant, scientific Kubernetes cluster," *arXiv preprint arXiv:2505.22864*, 2025.

[2] "NATS: Connective technology for adaptive edge and distributed systems," https://nats.io, accessed 2026.

[3] A. Zandieh, M. Daliri, M. Hadian, and V. Mirrokni, "TurboQuant: Online vector quantization with near-optimal distortion rate," *arXiv preprint arXiv:2504.19874*, 2025.

[4] M. Milakov and N. Gimelshein, "Online normalizer calculation for softmax," *arXiv preprint arXiv:1805.02867*, 2018.

[5] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[6] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.

[7] D. Thain, T. Tannenbaum, and M. Livny, "Distributed computing in practice: the Condor experience," *Concurrency and Computation: Practice and Experience*, vol. 17, no. 2-4, pp. 323–356, 2005.

[8] R. Chard, Y. Babuji, Z. Li, T. Skluzacek, A. Woodard, B. Blaiszik, I. Foster, and K. Chard, "funcX: A federated function serving fabric for science," in *Proc. 29th Int. Symp. on High-Performance Parallel and Distributed Computing (HPDC)*, 2020.

[9] P. Moritz, R. Nishihara, S. Wang *et al.*, "Ray: A distributed framework for emerging AI applications," in *Proc. 13th USENIX Symp. on Operating Systems Design and Implementation (OSDI)*, 2018.

[10] M. Rocklin, "Dask: Parallel computation with blocked algorithms and task scheduling," in *Proc. 14th Python in Science Conference (SciPy)*, 2015.

[11] Y. Babuji, A. Woodard, Z. Li *et al.*, "Parsl: Pervasive parallel programming in Python," in *Proc. 28th Int. Symp. on High-Performance Parallel and Distributed Computing (HPDC)*, 2019.

[12] A. H. Bond, "nats-bursting: bursting ai workloads to a shared cluster over NATS," https://pypi.org/project/nats-bursting, zenodo: 10.5281/zenodo.20660069; accessed 2026.

[13] ——, "turboquant-pro: Pca-matryoshka + turboquant embedding/kv compression," https://pypi.org/project/turboquant-pro, zenodo: 10.5281/zenodo.20660087; accessed 2026.

[14] ——, "batch-probe: Oom-safe gpu batch sizing and thermal-aware control," https://pypi.org/project/batch-probe, zenodo: 10.5281/zenodo.21095035; accessed 2026.